

参考：

- 《使用Memory Profiler查看 Java 堆和内存分配》 [链接](#)
- 《LeakCanary 中文使用说明》 [链接](#)

内存优化

内存优化主要包括

- **内存溢出优化**：Bitmap 的优化、Thread 缓冲池的使用
- **内存泄漏优化**：造成内存泄漏的原因及解决办法
- **防止内存抖动**：for 循环和 onDraw 里避免创建对象
- **数据结构的优化**：HashMap 的优化、String 的优化、少用枚举
- **View 的优化**：ListView（OOM）、LayoutInflater（内存泄漏）、View 动画（内存泄漏）
- 其他：少开进程、尽量使用系统资源

防止内存抖动

内存抖动：短时间内频繁创建对象和回收，造成内存抖动（**频繁 GC 引发的**）

- 避免在 for 循环中创建对象（造成内存抖动）
- 避免在 onDraw 中创建对象（频繁执行，会延缓 UI 线程）
- 通常是由于 String 拼接造成的

数据结构优化

ArrayMap 和 SparseArray 是 Android 系统的 API，专为移动设备定制，在一定情况下取代 HashMap 而达到节省内存的目的

- **SparseArray** 必须指定 key 为 int，相比 HashMap 可以省 **30%** 的内存
- **ArrayMap** 在小数据规模下，相比 HashMap 可以省 **10%** 的内存；数据量达到 **1000 以上** 时，查询速度可能会变慢

在频繁进行字符串拼接的时候，用 **StringBuilder** 代替 String 进行

- String 会创建大量的临时对象

少用枚举：

- 在早期的 Android 版本中，枚举相比直接使用 int，包体积大了 10 倍
- 随着虚拟机的优化，现在枚举相比直接使用 int，包体积大了 2 倍

View 的优化

在使用 `ListView` 的时候，尽量去复用 `convertView`；或者采用 `RecyclerView` 去替代

`填充布局` 的时候，尽量使用 `Application` 的上下文

- `Application` 和 `Activity` 是不同的
- `LayoutInflater.from(Context context)`：这里我们使用的 `context`，就是填充 `View` 所使用的上下文
- 如果使用 `Activity`，那么填充的 `View` 会拥有 `Activity` 的引用，如果此时该 `View` 存在于一个静态对象中，则会造成内存泄漏

动画结束，及时清除动画

- 当 `View` 开启一个动画后，`View` 和 `Animation` 会互相持有引用，必须在动画结束的时候，及时调用 `View.clearAnimation` 方法移除动画

其他

尽量使用 `系统组件`、`系统图片`、甚至控件的 `id`：例如 `@android:color/xxx`，`@android:style/xxx`

少开进程，即使是空进程也会占用 1M 左右的内存

内存分析工具

如果怀疑某个类，那么去跟踪它，看内存中是否存在多份拷贝，如果存在多份拷贝，**跟踪** 并找到可能造成内存泄露的地方

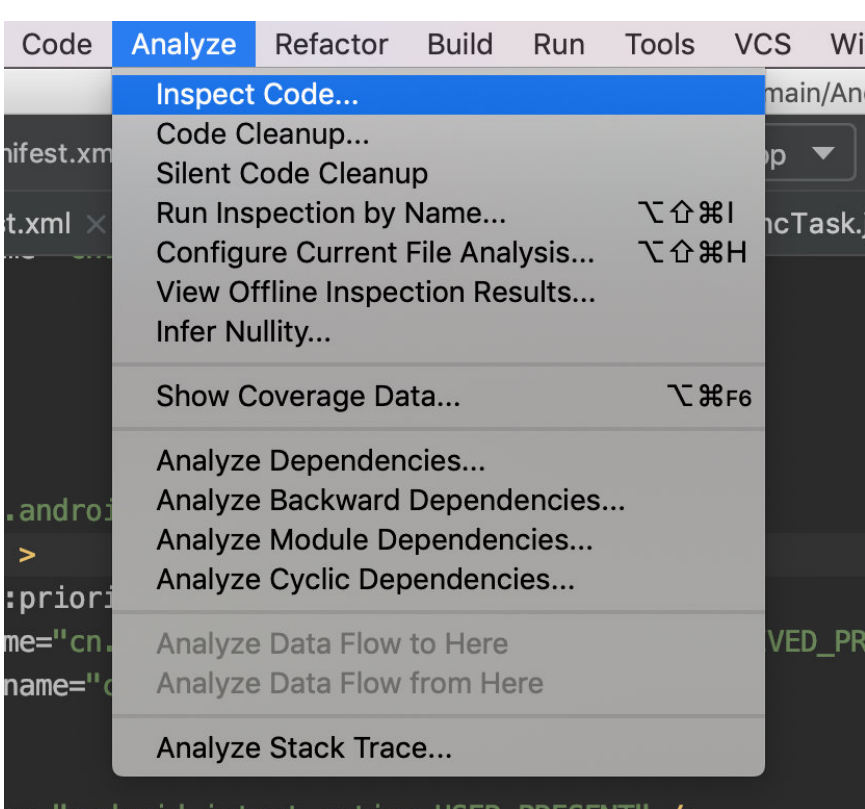
如果是整个项目，那么直接去查看内存中 **哪些类存在多份拷贝**，那么这些类就是有可能造成内存泄露的地方

跟踪的时候可以采用 **暴力置空** 的方式去检查

Lint 代码检查

Lint 是 Android Studio 自带的代码静态检测工具

打开：Analyze – Inspect Code – 然后选择扫描区域即可

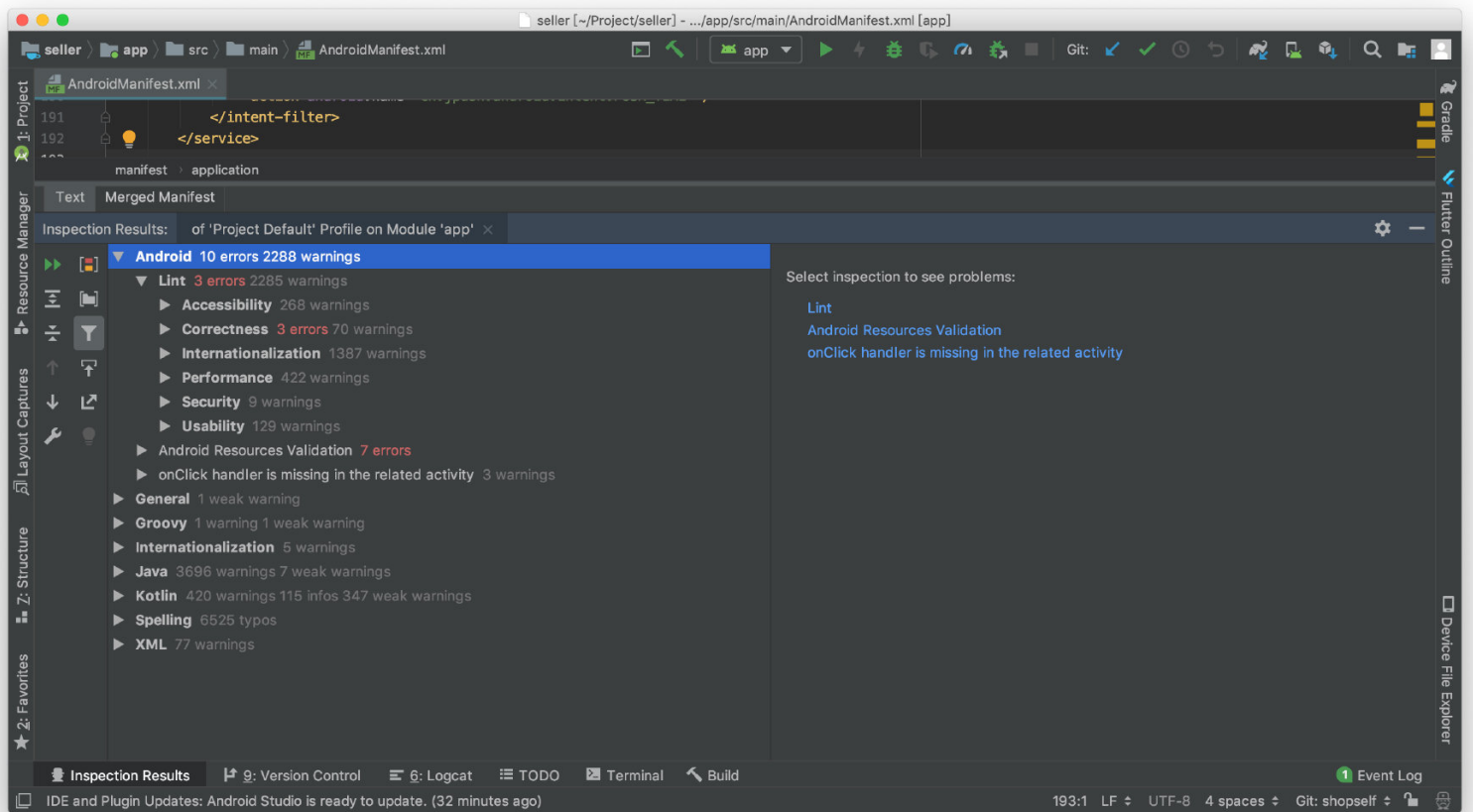


对有可能引起问题对代码 Link 都会进行提示，包括性能问题、安全问题、过时 API、格式编码问题等等都会进行提示

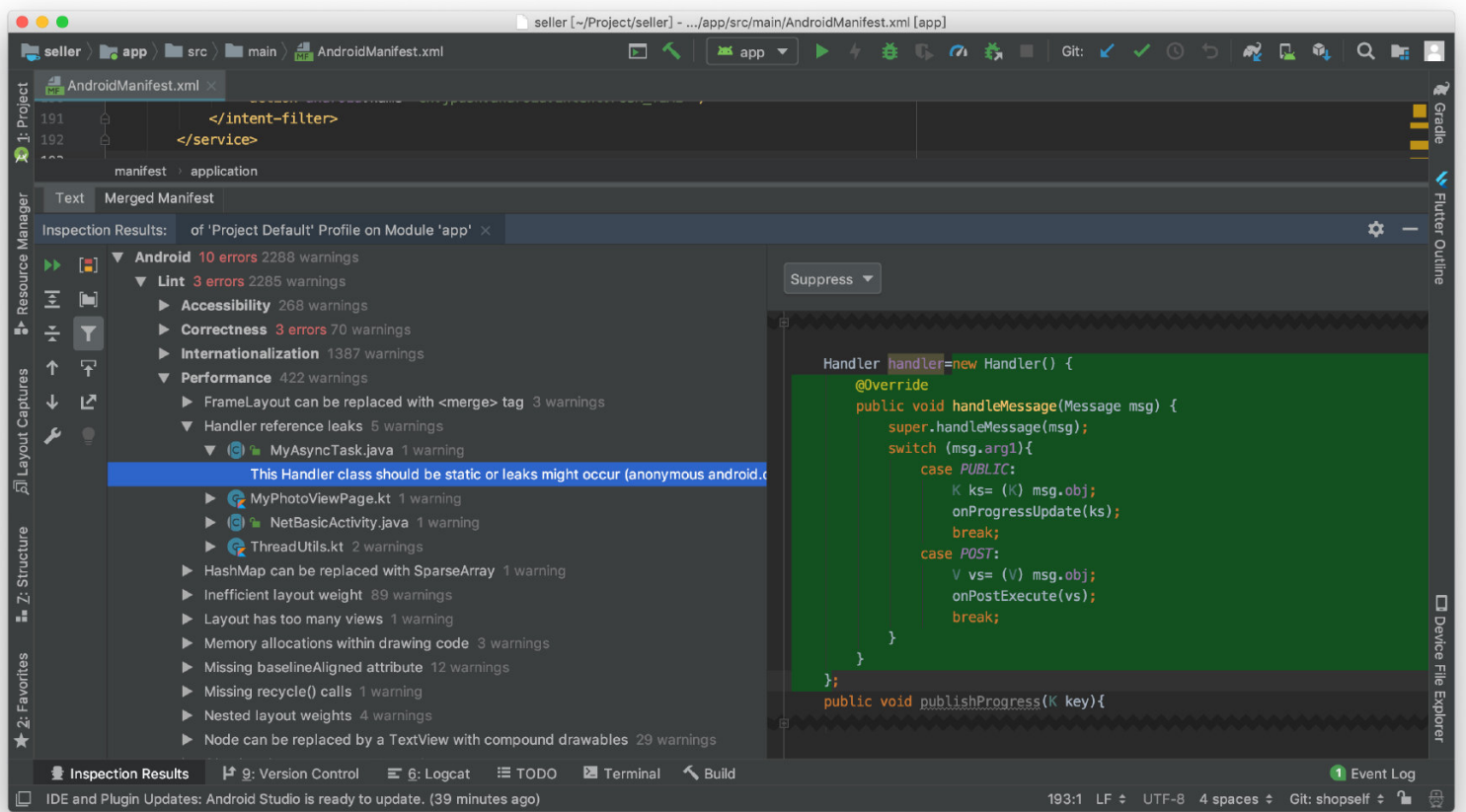
Performance：一些性能问题

Correct：一些 API 调用、编码格式问题等等

Security: 安全问题



这边显示的是 Handler 有可能造成内存泄漏



Monitor Memory 内存检测

Monitor Memory 是 Android Studio 自带的内存检测工具

- 我们可以利用 **Memory Profile 时间线**，查看可能导致内存泄漏的方法
- 再利用 **堆转储** 功能，查看可能导致内存泄漏的类和对象

怎样配合 Monitor Memory 检测内存泄漏

- 旋转屏幕：从纵向转为横向，并且在不同 Activity 状态下反复操作
 - 旋转屏幕的时候，系统会自动重建 Activity，容易导致

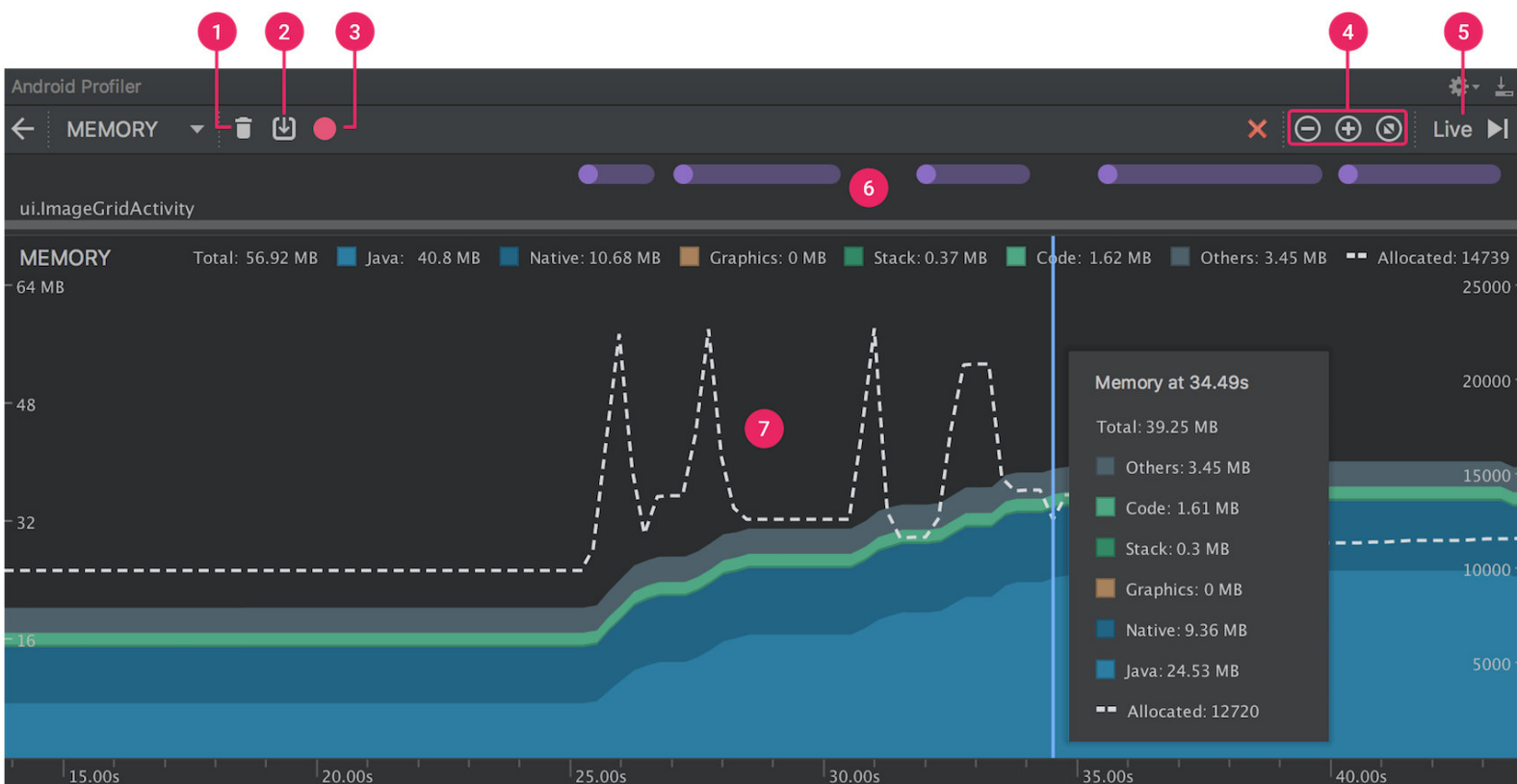
Activity、Context、View 对象的泄漏

- 在不同 Activity 状态下，反复在不同应用之间切换

检查步骤

1. 观察概览图，如果内存一直居高不下，那么就有可能发生了内存泄露、OOM 隐患
2. 查看内存分配情况：哪些对象占有了大量内存、哪些类存在多份实例
3. 查看内存泄露情况：哪些实例应当回收的，却依然出现在列表中，然后再去查询引用链表

– Memory Profile 概览



按钮概览：

1. **强制垃圾回收**：Force Garbage Collection
2. **堆转储**（将堆中的内存信息转为本地存储文件）：Dump Java Heap
3. **记录内存分配**：Record Memory Allocation
4. 放大/缩小/复原时间线
5. 跳转至实时内存数据
6. **Event 时间线**：记录当前 Activity 的状态、以及点击、跳转、屏幕旋转等Event事件
7. **Memory 时间线**

内存类别

- Java：从 Java / Kotlin 代码分配的内存（包括了 Zygote 启动开始所需的所有Java内存）
- Native：从 C / C++ 代码分配的内存（包括一些原生内存）
- Graphics：图形图像内存（这里是与CPU共享的内存，而不是GPU专用内存）
- Stack：栈内存（包括原生堆栈和Java堆栈）；与APP运行的线程有关
- Code：处理代码和资源的内存（dex字节码、so库、字体等）
- Others：其他
- Allocated：从 Java / Kotlin 代码分配的 对象数
 - 需要注意的是，这里统计的对象数：
 - Android 7.1 及其之前的版本，是当 Memory Profile 连接

至 APP 时候才开始统计的；

- Android 8.0 开始，因为自带了一个设备内置分析工具（该工具可以记录所有分配），统计的是 APP 中所有待回收的对象数

- 记录内存分配

点击 **Record Memory Allocation** 按钮，可以记录一段时间内的内存分配情况

- 记录内存分配，没啥用，通常配合堆转储，获取堆叠追踪
- 记录内存分配，通常用来检测 **OOM**

我们可以利用该功能，查看占用内存最大的对象，然后对该对象进行优化（缩小、内存泄露等等）

需要注意的是：**这里记录的是这段时间内被创建过的所有对象**

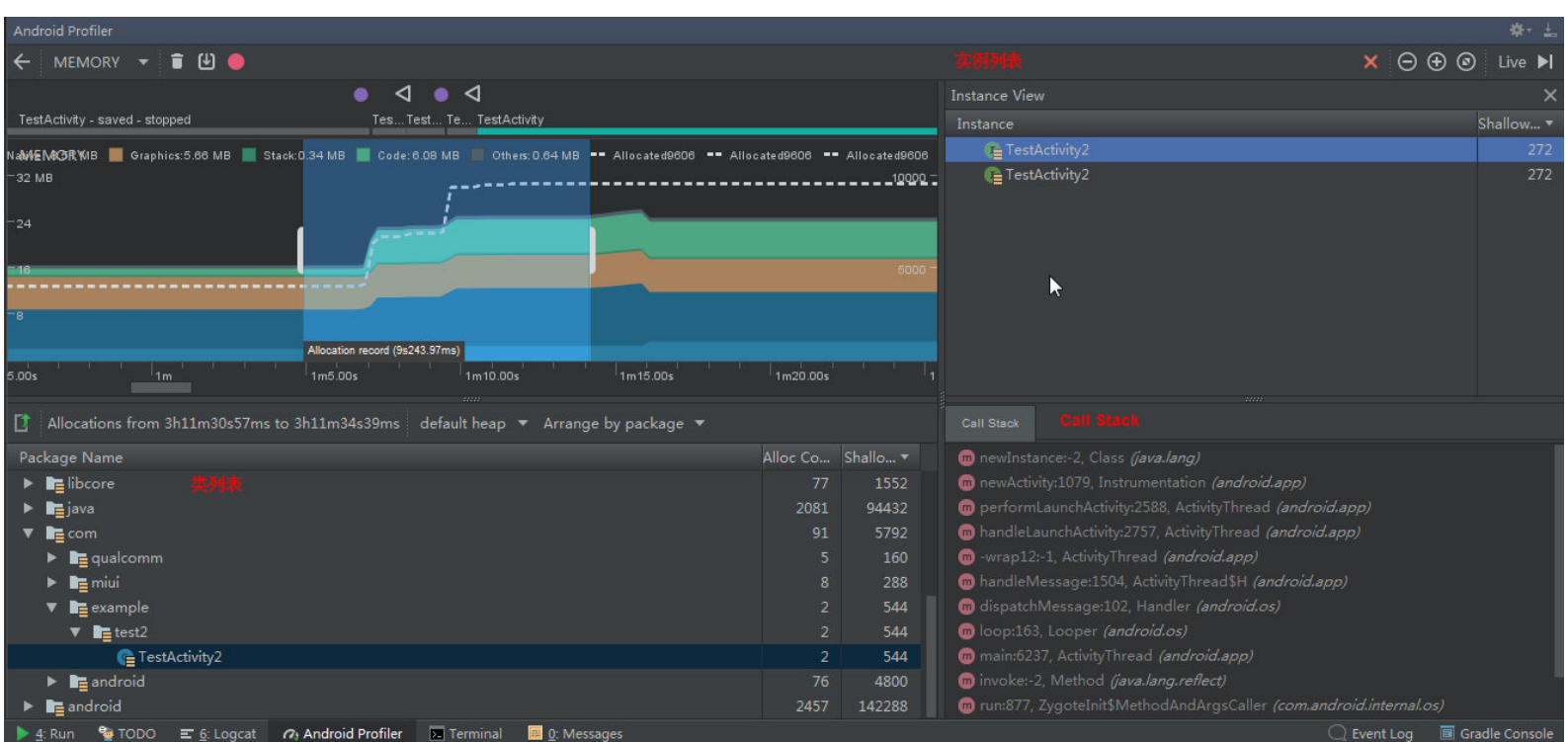
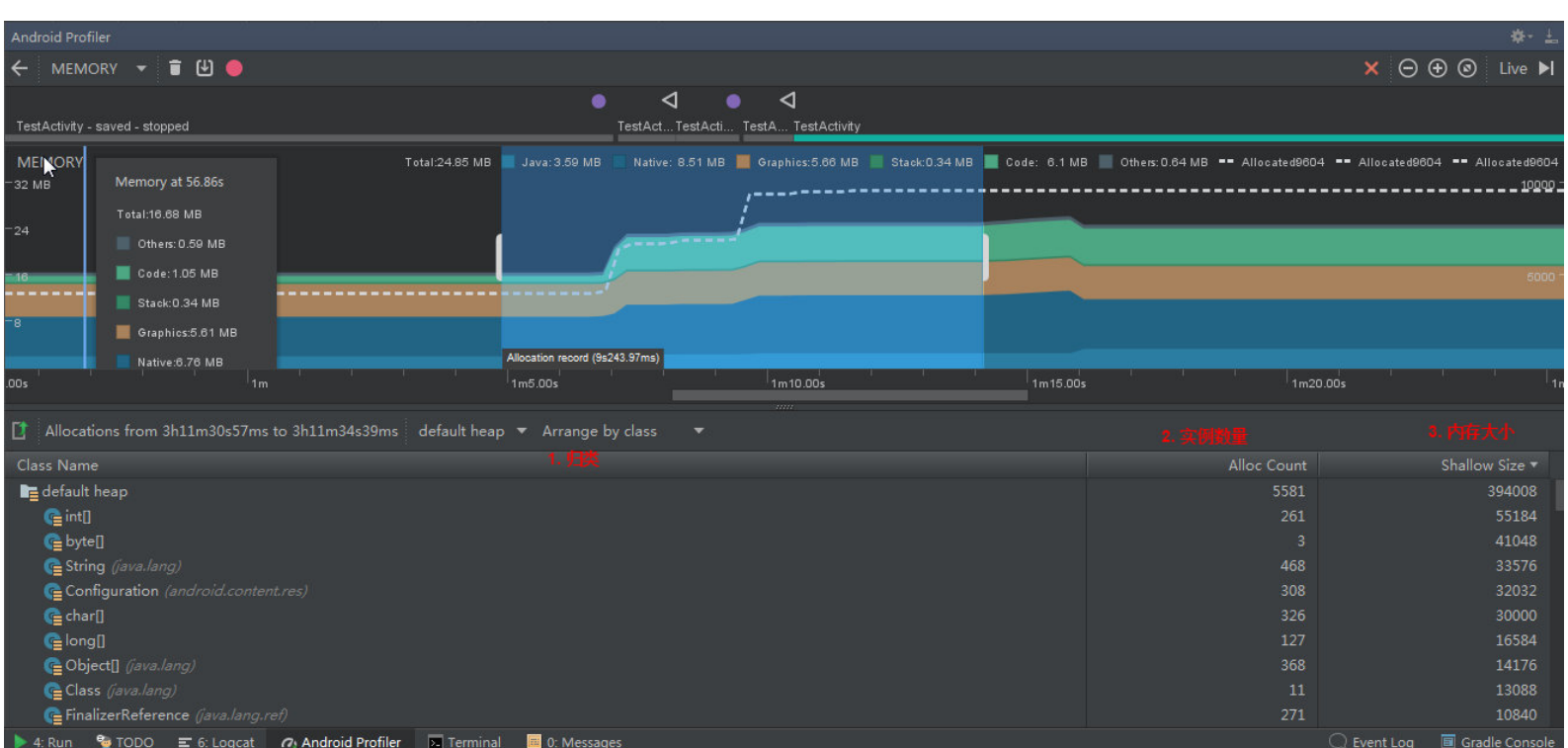
- 这段时间内被创建出来的对象：在这之前已经存在的对象是不会记录进来的
- 这段时间内在内存中存在过的所有对象：在这段时间内，某个对象，被创建出来、又被销毁了，也会出现在列表中

类列表：点击这块区域内的任意一点，在时间线的下方会列出，这段时间内，内存中所创建出的所有类列表

- **Alloc Count**: 此类的对象数量
- **Shallow Size**: 此类的内存大小 (此刻在内存中占用的大小)
- **Retained Size**: 此类的所有实例而在内存中保留的大小 (历史总合)
- 可以对列表中的类进行归类
 - Arrange by class: 按类进行归类
 - Arrange by package: 按包名进行归类
 - Arrange by callstack: 按调用堆栈进行归类

对象列表: 我们也可以点击其中一个类, 会在右侧弹出 **Instance View**; 点击其中一个类, 会在下面弹出 **Call Stack**

- Instance View: 类所对应的所有 **实例列表**
- Call Stack: **堆叠追踪**; 显示具体实例被分配到何处、以及所在线程



- 堆转储

堆转储 **Dump Java Heap**：显示某一个时刻（捕获堆转储的那一刻）**在内存中存在的所有对象**

- 注意，它与“记录内存分配”的区别是：记录内存分配显示的

是某一个时间段内在内存中 **创建过** 的 **所有对象**

- 在点击堆转储按钮的那一刻，Java内存会有所增加，因为堆转储与APP在同一进程，需要分配一定的内存来收集数据

堆转储，通常用来 **检测内存泄漏**

- 我们在堆转储之前，点击“强制GC”按钮，让系统进行垃圾回收
- 然后在堆转储的时候，我们可以看到哪些理应回收的对象，仍然存在内存中，这些就是内存泄漏的对象
- “记录内存分配”无法检测内存泄漏

类列表

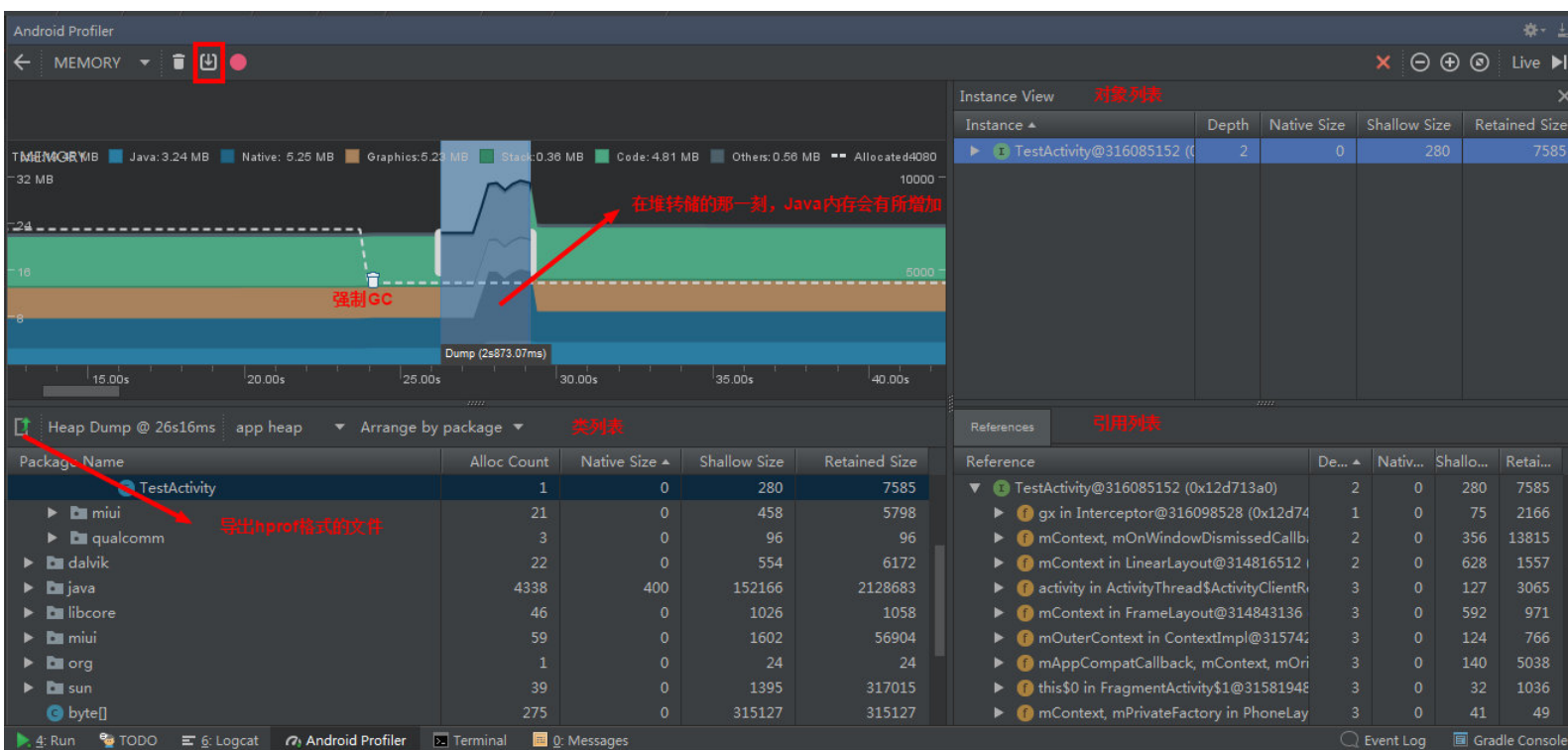
- Alloc Count: 此类的对象数量（通常为 1，否则可能造成了内存泄露）
- Shallow Size: 此类的内存大小（此刻在内存中占用的大小）
- Retained Size: 为此类的所有实例而在内存中保留的大小（历史总合）

对象列表

- Shallow Size: 此对象的内存大小
- Retained Size: 此对象支配的内存大小
- **Depth**: 从 GC根 到此对象的 hop 数

引用列表

- 引用列表，默认不会展示每个对象的堆叠追踪；需要我们在堆转储之前，点击“记录内存分配”，然后在引用列表中就会出现Call Stack选项
- 包含堆叠追踪的对象实例会有一个图标 



- HPROF

将堆转储文件，导出为 **hprof 文件** 后，我们可以随时随地的查看堆转储，同时我们还可以借助第三方工具进行查看分析

- 第三方工具：**jhat**
- 在借助第三方 HPROF 分析器的时候，我们需要将 HPROF 文件从 Android 格式转为 **Java SE HPROF 格式**

- 我们可以利用 `android_sdk/platform-tools` 目录中提供的 `hprof-conv` 工具执行转换操作 `hprof-conv -z heap-original.hprof heap-converted.hprof`
`-z` 表示只选取 `appHeap`

利用 Android Studio 打开 HPROF 文件，我们除了可以查看之前堆转储的这些信息外，还可以利用 **Analyzer Tasks** 工具，分析内存中可能存在泄漏的 Activity

The screenshot displays the Android Studio interface for memory analysis. The 'App heap' tab is active, showing a table of class names and their memory usage. The 'Analyzer Tasks' panel on the right shows 'Detect Leaked Activities' and 'Find Duplicate Strings' tasks. The 'Analysis Results' section lists 'Leaked Activities' with details for TestActivity2 instances.

Class Name	Total	Heap	Size of	Shallow	Retained
com	384	0	0	258723	
example	9	0	0	154422	
test2	9	0	0	154422	
TestActivity2 (com.example.test2)	4	4	272	1088	153169
TestActivity (com.example.test2)	1	1	280	280	1205
TestActivity\$1 (com.example.test2)	4	4	12	48	48
android	316	0	0	100089	
miui	47	0	0	3828	
qualcomm	12	0	0	384	
char[]	52684	6022	0	18094	180942
long[]	5186	1798	0	12718	127184
int[]	6500	4063	0	11816	118164

The 'Analyzer Tasks' panel shows the following tasks:

- ☒ Detect Leaked Activities
- ☒ Find Duplicate Strings

The 'Analysis Results' section shows the following results:

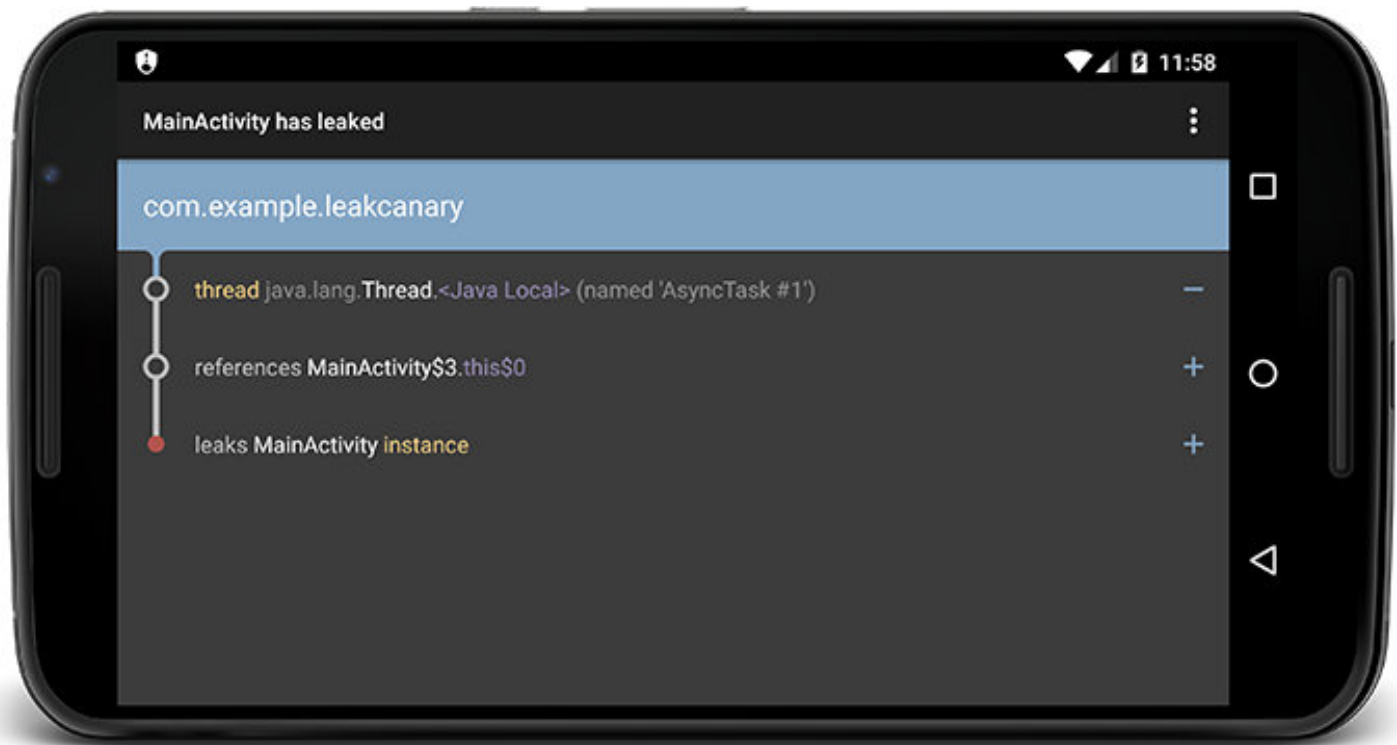
- Duplicated Strings**
- Leaked Activities**
 - 0 = (TestActivity2@317818208 (0x12f18560))
 - 1 = (TestActivity2@317817392 (0x12f18230))
 - 2 = (TestActivity2@316575200 (0x12de8de0))
 - 3 = (TestActivity2@316574112 (0x12de89a0))

LeakCanary

LeakCanary 是一个安装在手机中的 APP，它可以帮我们自动检测是否存在内存泄漏

- 采用三方依赖的形式进行安装，它会依附在我们的 APP 中

- 一旦发生内存泄漏，LeakCanary 就会将泄漏信息显示在 APP 中



- 使用

1.集成

```
debugImplementation 'com.squareup.leakcanary:leakcanary-android:1.6.1'
releaseImplementation 'com.squareup.leakcanary:leakcanary-android-no-op:1.6.1'
// 检测Fragment的内存泄漏
debugImplementation 'com.squareup.leakcanary:leakcanary-support-fragment:1.6.1'
```

2.初始化

调用 `LeakCanary.install()` 方法进行初始化，初始化动作放在 Application 中

```
if (LeakCanary.isInAnalyzerProcess(this)) {  
    return;  
}  
LeakCanary.install(this);
```

3.之后 LeakCanary 就可以自动检测 Activity 的内存泄漏了

- **自动检测**：Activity、Fragment 的内存泄漏，在集成 LeakCanary 成功后，即可自动检测
- **手动检测**：我们可以利用 `RefWatcher` 对象，手动检测某个类的内存泄漏情况 `RefWatcher.watch()`

```
RefWatcher refWatcher = LeakCanary.install(this);  
  
public abstract class BaseFragment extends Fragment {  
  
    @Override public void onDestroy() {  
        super.onDestroy();  
        refWatcher.watch(this);  
    }  
  
}
```

- 上传到服务器

LeakCanary 还支持将内存泄漏信息上传到服务器

1.创建一个 LeakUploadService, 继承自

DisplayLeakService

```
// 定义 Service
public class LeakUploadService extends DisplayLeakService {
    @Override
    protected void afterDefaultHandling(HeapDump heapDump,
        AnalysisResult result, String leakInfo) {
        if (!result.leakFound || result.excludedLeak) {
            return;
        }
        myServer.uploadLeakBlocking(heapDump.heapDumpFile,
            leakInfo); // 上传操作
    }
}

// 注册 Service
<application
    android:name="com.example.DebugExampleApplication">
    <service android:name="com.example.LeakUploadService" />
</application>
```

2.自定义 RefWatcher, 参数为 DisplayLeakService

```
RefWatcher refWatcher = LeakCanary.install(app,
    LeakUploadService.class);
```